

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE TRABAJO PRÁCTICO

INFORMACIÓN BÁSICA					
ASIGNATURA:	LABORATORIO PROGRAMACIÓN WEB 2				
TÍTULO DEL TRABAJO:	LAB 03				
NÚMERO DE TRABAJO:	03	AÑO LECTIVO:	2023	NRO. SEMESTRE:	1°
FECHA DE PRESENTACIÓN	22/05/23	HORA DE PRESENTACIÓN	hh/mm/ss		
INTEGRANTE (s)	Sebastian Arley Chirinos Negrón			NOTA (0-20)	
DOCENTE(s):	RICHART SMITH ESCOBEDO QUISPE				

INTRODUCCIÓN
Lista (11 pts) Implementa una Lista usando POO con tipos genéricos siguiendo los estándares de Java. (Los métodos para una lista). https://docs.oracle.com/javase/7/docs/api/java/util/List.html Puede ignorar los siguientes métodos: hashCode() iterator() listIterator() listIterator(int index) retainAll(Collection<?> c) toArray() toArray(T[] a)

```
1  public class MyList<T> {  
    8 usages  
2      Node<T> root; // referencia al nodo inicial  
    5 usages  
3      private int size; // tamaño de la lista  
4  
5      // Constructor  
    no usages  
6      public MyList() {  
7          root = null;  
8          size = 0;  
9      }  
10  
11     // Obtener la longitud de la lista  
    no usages  
12     public int getSize() {  
13         return size;  
14     }  
15  
16     // Verificar si la lista está vacía  
    1 usage  
17     public boolean isEmpty() {  
18         return size == 0;  
19     }  
20  
21     // Imprimir la lista  
    no usages  
22     public void printList() {  
23         Node<T> current = root;  
24         while (current != null) {
```

```
25         System.out.print(current.getData() + " ");
26         current = current.getNextNode();
27     }
28     System.out.println();
29 }
30
31 // Añadir un elemento al final de la lista
no usages
32 public void add(T data) {
33     Node<T> newNode = new Node<>(data);
34     if (isEmpty()) {
35         root = newNode;
36     } else {
37         Node<T> current = root;
38         while (current.getNextNode() != null) {
39             current = current.getNextNode();
40         }
41         current.setNextNode(newNode);
42     }
43     size++;
44 }
45
46 // Obtener el elemento en un índice específico
no usages
47 public T get(int index) {
48     Node<T> current = root;
49     for (int i = 0; i < index; i++) {
50         current = current.getNextNode();
51     }
52     return current.getData();
```

```
53     }  
54  
55     // Eliminar el elemento en un índice específico  
no usages  
56     public void remove(int index) {  
57         if (index == 0) {  
58             root = root.getNextNode();  
59         } else {  
60             Node<T> current = root;  
61             for (int i = 0; i < index - 1; i++) {  
62                 current = current.getNextNode();  
63             }  
64             current.setNextNode(current.getNextNode().getNextNode());  
65         }  
66         size--;  
67     }  
68 }  
69
```

(i) Implemente una clase Node donde T es un tipo genérico, esta clase debe contener al menos dos propiedades.

<https://docs.oracle.com/javase/tutorial/java/generics/types.html>

T data: la información almacenada en el nodo

Node<T> nextNode: una referencia al siguiente nodo

(ii) Implementar una clase List esta clase debe contener al menos esta propiedad

Node<T> root: la referencia sobre el nodo inicial

```
1  public class Node<T> {  
    3 usages  
2      private T data; // Almacena el valor de datos del nodo  
    3 usages  
3      private Node<T> nextNode; // Referencia al siguiente nodo en la lista  
4  
5      // Constructor que recibe el valor de datos del nodo  
    1 usage  
6      public Node(T value) {  
7          data = value;  
8          this.nextNode = null; // Inicialmente,  
9          // el nodo no tiene un  
10         // siguiente nodo, por lo que se establece como null  
11     }  
12  
13     // Método para obtener el valor de datos del nodo  
    2 usages  
14     public T getData() {  
15         return data;  
16     }  
17  
18     // Método para establecer el valor de datos del nodo  
    no usages  
19     public void setData(T value) {  
20         this.data = value;  
21     }  
22  
23     // Método para obtener la referencia al siguiente nodo  
    8 usages
```

```

23      // Método para obtener la referencia al siguiente nodo
      8 usages
24      public Node<T> getNextNode() {
25          return nextNode;
26      }
27
28      // Método para establecer la referencia al siguiente nodo
      2 usages
29      public void setNextNode(Node<T> nextNode) {
30          this.nextNode = nextNode;
31      }
32  }
```

CUESTIONARIO

¿Qué diferencia hay entre un List y un ArrayList en Java?

List: List es una interfaz que define las operaciones básicas que se pueden realizar en una lista, como agregar, eliminar, acceder a elementos, entre otros. Es una interfaz de nivel superior y no se puede instanciar directamente. Al ser una interfaz, puede ser implementada por diferentes clases, incluyendo ArrayList.

ArrayList: ArrayList es una clase concreta que implementa la interfaz List. Proporciona una implementación basada en un array dinámico, lo que significa que el tamaño del ArrayList puede crecer dinámicamente según sea necesario. Esto permite agregar y eliminar elementos de manera eficiente, así como acceder rápidamente a los elementos por su índice. Sin embargo, esta eficiencia puede verse afectada si se realizan operaciones frecuentes de inserción o eliminación en posiciones intermedias de la lista.

¿Qué beneficios y oportunidades ofrecen las clases genéricas en Java?

Reutilización de código: Las clases genéricas permiten escribir código que pueda trabajar con diferentes tipos de datos, lo que promueve la reutilización del código en diferentes contextos y evita la necesidad de duplicar código para tratar tipos diferentes.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

Seguridad en tiempo de compilación: El uso de clases genéricas proporciona seguridad en tiempo de compilación al especificar el tipo de datos que se utilizará en la clase. Esto permite detectar y corregir errores de tipo en el momento de compilación, en lugar de descubrirlos en tiempo de ejecución.

Abstracción de datos: Las clases genéricas permiten crear estructuras de datos y algoritmos que no están acoplados a un tipo de dato específico, lo que proporciona una mayor abstracción y flexibilidad en el diseño del código.

REFERENCIAS

- <https://www.w3schools.com/java/>
- <https://www.eclipse.org/downloads/packages/release/2022-03/r/eclipse-ide-enterprise-java-and-web-developers>
- <https://docs.oracle.com/javase/7/docs/api/java/util/List.html>
- <https://docs.oracle.com/javase/tutorial/java/generics/types.html>